Clasificación de imágenes de anime, caricaturas y personas mediante redes neuronales convolucionales

Manuel Bajos Rivera, A01711581

Resumen

En este proyecto se realiza la clasificación automática de imágenes en tres categorías: anime, caricaturas (cartoon) y personas (human). Se utiliza el conjunto de datos Anime vs Cartoon vs Human de Kaggle, con 8,917 imágenes casi balanceadas entre las tres clases (3,004 anime, 2,913 cartoon y 3,000 human), de las cuales quedan 8,534 tras eliminar 383 duplicados exactos. El análisis exploratorio mostró además diferencias marcadas de resolución, peso de archivo y color entre clases, que podrían permitir al modelo clasificar por atajos y no por contenido.

Se implementó una red neuronal convolucional propia con TensorFlow y Keras como modelo base, y se comparó con transferencia de aprendizaje usando EfficientNetB0 preentrenada con ImageNet. Se probaron además, cada uno por separado, la regularización con Dropout, estirar las imágenes en lugar de rellenarlas, el aumento de datos, un aumento dirigido a la falla (alejar, recortar y borrar zonas, también por separado), imágenes de 224 $\times$ 224 píxeles, redes con 2 a 5 bloques convolucionales y distintas profundidades del ajuste fino de EfficientNetB0. Los modelos se evaluaron en validación, en prueba y con 8 imágenes externas al conjunto de datos.

En prueba, la CNN base alcanzó 99.53 % de exactitud y EfficientNetB0 99.92 %, pero todos los modelos superan el 99.5 % y esas métricas no permiten distinguirlos. Con las imágenes externas el desempeño cae de forma notable: la CNN base acierta 2 de 8 imágenes (25.0 %), EfficientNetB0 acierta 4 de 8 (50.0 %) y 5 de 8 (62.5 %) con aumento de datos, mientras que Dropout, estirar, el aumento dirigido, la mayor resolución y cambiar la profundidad de la CNN no mejoraron ese resultado. Entrenar solo la cabeza de EfficientNetB0, sin ajuste fino, obtuvo el mejor resultado externo: 6 de 8 con tres semillas, una imagen más que el ajuste de las últimas 20 capas. Esta configuración se volvió a entrenar y sus pesos se guardaron para usarla en la interfaz web. Ningún modelo acierta a la vez las dos ilustraciones anime de estilo moderno, y la mayoría de los errores se clasifican como cartoon. Con un conjunto externo tan pequeño las diferencias no son concluyentes, pero indican que la exactitud en validación y prueba no se mantiene con imágenes de otro origen, lo que podría deberse a la poca diversidad de los datos de entrenamiento. Los modelos pueden probarse mediante una interfaz web con selección de modelo.

1. Problemática

Distinguir automáticamente el estilo visual de una imagen (dibujo de anime, caricatura occidental o fotografía de una persona) es un problema de clasificación de imágenes que puede resultar útil, por ejemplo, para organizar colecciones de imágenes o filtrar contenido. Las tres categorías comparten elementos como rostros, colores y contornos, pero difieren en el trazo, la textura y la composición, por lo que no es una tarea trivial para un clasificador basado en reglas.

Dado esto, se plantea un problema de clasificación multiclase cuya meta es predecir, a partir de una imagen a color, a cuál de las tres clases pertenece: `anime`, `cartoon` o `human`. Las variables de entrada son los píxeles de la imagen y la variable objetivo es la etiqueta de la clase.

Dado que los datos son imágenes, se emplean redes neuronales convolucionales (CNN), que son capaces de aprender de forma automática jerarquías de características visuales a partir de los píxeles.

2. Datos

El conjunto de datos utilizado corresponde a Anime vs Cartoon vs Human, disponible en Kaggle [1]. Contiene 8,917 imágenes organizadas en tres carpetas, una por clase. La Tabla 1 resume su composición.

Cuadro 1. Composición del conjunto de datos original

Clase	Imágenes	Formato	Ancho típico	Alto típico	Peso típico
anime	3,004	JPEG	68 px	68 px	4 KB
cartoon	2,913	PNG y JPEG	258 px	180 px	55 KB
human	3,000	PNG	1,024 px	1,024 px	1,334 KB

La Figura 1 muestra que las clases están casi balanceadas, con entre 2,913 y 3,004 imágenes por clase, por lo que la exactitud (accuracy) es una métrica adecuada para comparar los modelos. Sin embargo, como la exactitud global puede ocultar un desempeño deficiente en una clase particular, se reportan además la precisión, la sensibilidad (recall) y la medida F1 por clase, así como el F1 macro, que promedia las tres clases con el mismo peso, y la matriz de confusión.

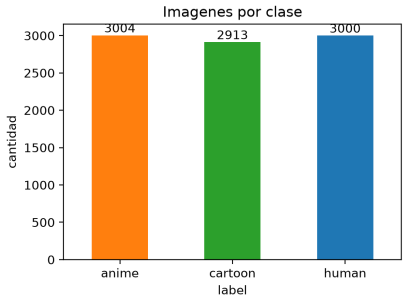

Figura 1. Número de imágenes por clase en el conjunto original.

Las imágenes tienen características muy distintas entre clases. Las imágenes de anime son miniaturas cuadradas de entre 25 y 189 píxeles de ancho. Las de cartoon van de 74 a 519 píxeles de ancho y su relación de aspecto varía entre 0.37 y 5.35, de modo que solamente el 12 % son cuadradas. Las de human son siempre cuadradas de 1,024 × 1,024 píxeles.

Además, las imágenes de cartoon mezclan tres modos de color: 653 en RGB, 2,186 en RGBA y 74 con paleta indexada. Ninguna de las imágenes con canal alfa presenta transparencia real, por lo que su conversión a RGB no altera el contenido.

3. Análisis exploratorio

Antes del entrenamiento se realizó un análisis de la integridad, la resolución, el color y el contenido de las imágenes. Se verificó que ninguna de las 8,917 imágenes estuviera corrupta.

La Figura 2 presenta una muestra aleatoria de imágenes de cada clase, con su resolución original. Se observa que tanto anime como human son retratos de rostros recortados, mientras que cartoon incluye personajes de cuerpo completo, escenas y capturas de pantalla con márgenes. Esto indica que, además del estilo, la composición de la imagen también diferencia a las clases.

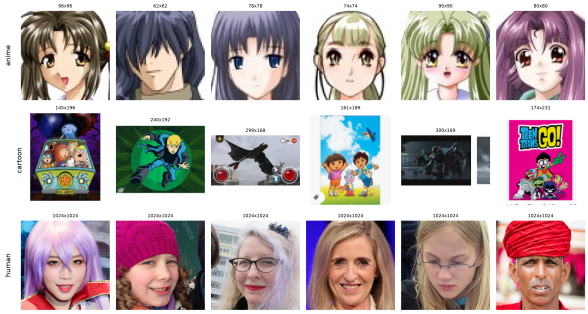

Figura 2. Muestra aleatoria de imágenes por clase. El título de cada imagen indica su resolución original.

3.1. Riesgo de atajos por resolución y peso

La Figura 3 compara el valor típico (mediana) de cada clase. Las imágenes de human son aproximadamente 15 veces más anchas y 300 veces más pesadas que las de anime. Esta diferencia es tan marcada que, por sí sola, permitiría identificar la clase sin observar el contenido de la imagen.

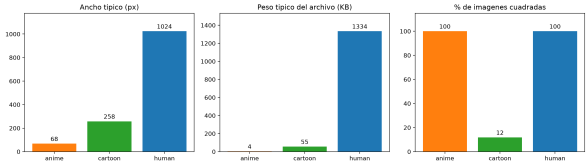

Figura 3. Ancho típico, peso típico del archivo y porcentaje de imágenes cuadradas por clase.

Esto representa un riesgo, ya que un modelo podría aprender un atajo basado en la nitidez o la resolución y no en las características visuales de cada categoría, lo que produciría una exactitud alta pero engañosa. Para mitigarlo, todas las imágenes se redimensionan a un mismo tamaño antes de entrenar, y se evalúa el modelo con imágenes externas que no provienen del conjunto de datos.

Una diferencia similar aparece en el color. La Figura 4 muestra que las imágenes de human son en promedio más oscuras (medias de los canales R, G y B de 0.52, 0.42 y 0.38) que las de anime (0.64, 0.54 y 0.51) y cartoon (0.61, 0.58 y 0.55), lo que constituye otra señal que el modelo podría aprovechar.

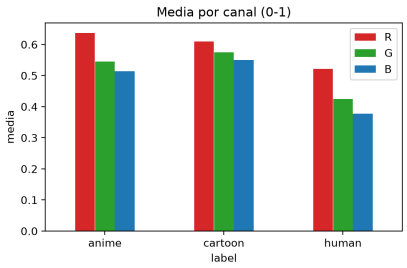

Figura 4. Media de cada canal de color por clase, sobre una muestra de 300 imágenes por clase.

3.2. Imágenes duplicadas

Se calculó el resumen criptográfico MD5 del contenido de cada archivo para detectar duplicados exactos. Se encontraron 712 imágenes involucradas en 329 grupos de duplicados, es decir, 383 imágenes sobrantes. Todas pertenecen a la clase anime y ningún grupo mezcla clases distintas. La Figura 5 muestra algunos ejemplos: son la misma imagen guardada con nombres de archivo diferentes. La mayoría de los grupos son pares, aunque algunos contienen hasta cinco copias.

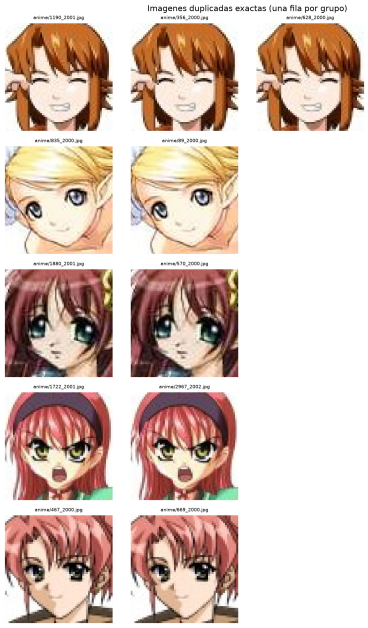

Figura 5. Ejemplos de grupos de imágenes duplicadas exactas (una fila por grupo).

Los duplicados deben eliminarse antes de dividir los datos. De lo contrario, una copia podría quedar en el conjunto de entrenamiento y otra en el de prueba, y el modelo habría visto la respuesta durante el entrenamiento, con lo que el desempeño en prueba sería artificialmente alto. Tras la deduplicación el conjunto contiene 2,621 imágenes de anime, 2,913 de cartoon y 3,000 de human (8,534 en total), con un desbalance leve.

4. ETL

El proceso de extracción, transformación y carga se realizó antes del entrenamiento con el objetivo de obtener un conjunto de datos adecuado para la clasificación. El proyecto se implementó en Python con TensorFlow 2.20 y Keras 3 [2, 3], versiones que cuentan con soporte en macOS, Linux y Windows.

La extracción se hizo mediante la biblioteca `kagglehub`, que descarga el conjunto de datos y lo guarda en una caché local. Posteriormente se construyó una tabla con la ruta y la clase de cada imagen, y se eliminaron los duplicados exactos como se describió en la sección anterior.

4.1. Partición de los datos

Sobre las imágenes sin duplicados se realizó una partición estratificada por clase en tres subconjuntos: 70 % para entrenamiento, 15 % para validación y 15 % para prueba, con una semilla fija (42) para que los resultados sean reproducibles. La estratificación mantiene en cada subconjunto la misma proporción de clases que en el total, lo que conserva el leve desbalance de anime. La Tabla 2 muestra el resultado.

Cuadro 2. Número de imágenes por clase y partición

Clase	Entrenamiento	Validación	Prueba	Total
`anime`	1,834	393	394	2,621
`cartoon`	2,039	437	437	2,913
`human`	2,100	450	450	3,000
Total	5,973	1,280	1,281	8,534

La partición se hizo sobre las rutas de las imágenes y antes de transformarlas, de modo que ninguna información de validación o prueba interviene en el entrenamiento. Además, se verificó mediante el resumen MD5 que ninguna imagen aparece en más de una partición. El conjunto de entrenamiento se utiliza para ajustar los parámetros de los modelos, el de validación para monitorear el desempeño y ajustar hiperparámetros, y el de prueba se mantiene aislado y se usa únicamente para reportar el desempeño final.

4.2. Transformación de las imágenes

Cada imagen se convierte a RGB de tres canales y se deja en un tamaño único de 128×128 píxeles, ya

que la red convolucional requiere entradas de dimensiones fijas. El redimensionado se realiza con el filtro de Lanczos de la biblioteca Pillow [4], un método de remuestreo por convolución con antialiasing, aplicado de la misma manera a todas las clases. Esto es relevante porque reducir imágenes sin un filtro adecuado puede introducir aliasing [5], y en este conjunto de datos la reducción es considerable, por ejemplo de 1,024 a 128 píxeles en las imágenes de human.

Como las imágenes de cartoon no son cuadradas en su mayoría, para llegar a un cuadrado se utiliza relleno (padding): se reduce la imagen conservando su proporción, de modo que su lado mayor mida 128 píxeles, y se centra sobre un lienzo cuadrado rellenado con el color mediano de su propio borde, para que las bandas resultantes sean poco llamativas. De esta forma no se deforman las imágenes.

Sin embargo, como solo el 12 % de las imágenes de cartoon son cuadradas y el 100 % de anime y human lo son, el relleno hace que casi todas las imágenes de cartoon tengan bandas y ninguna de las otras clases, lo cual puede convertirse en un atajo para el modelo. Por ello, en una sección posterior se compara con la alternativa de estirar la imagen a un cuadrado.

4.3. Carga y almacenamiento en caché

Las imágenes transformadas se guardan como arreglos de enteros sin signo de 8 bits (`uint8`) en una caché local, uno por partición. De esta forma, el costo de decodificar y reducir las imágenes originales, en particular las de 1,024 × 1,024 píxeles de human, se paga una sola vez. El conjunto de entrenamiento ocupa aproximadamente 294 MB en memoria, por lo que cabe completo en la RAM. El procesamiento de las 8,534 imágenes se paraleliza con hilos y tomó 11 segundos en la computadora utilizada. Las etiquetas se codifican como enteros (0 para anime, 1 para cartoon y 2 para human).

La normalización de los píxeles al rango $[0,1]$ se realiza dividiendo entre 255 mediante una capa `Rescaling` dentro de la CNN propia. Para la arquitectura preentrenada (EfficientNetB0) no se divide entre 255: la versión de Keras incluye en sus primeras capas el reescalado y la normalización de la entrada, y espera píxeles en el rango $[0,255]$, que es como se guardan las imágenes en la caché.

5. Modelos

5.1. CNN propia (modelo base)

Como primer modelo se diseñó una red neuronal convolucional sencilla y se entrenó desde cero, sin regularización ni aumento de datos, para tener un punto de comparación claro. Cada mejora posterior se prueba por separado, de modo que se pueda explicar qué cambio funcionó y por qué.

La red recibe imágenes RGB de 128×128 píxeles y está formada por tres bloques convolucionales, cada uno con una capa convolucional y una de agrupación máxima, seguidos de un clasificador denso. La Tabla 3 muestra sus capas.

Cuadro 3. Arquitectura de la CNN base

Capa	Salida	Parámetros
Reescalado (÷255)	$128 \times 128 \times 3$	0
Conv2D, 32 filtros 3×3, ReLU	$126 \times 126 \times 32$	896
MaxPooling2D 2×2	$63 \times 63 \times 32$	0
Conv2D, 64 filtros 3×3, ReLU	$61 \times 61 \times 64$	18,496
MaxPooling2D 2×2	$30 \times 30 \times 64$	0
Conv2D, 128 filtros 3×3, ReLU	$28 \times 28 \times 128$	73,856
MaxPooling2D 2×2	$14 \times 14 \times 128$	0
Flatten	25,088	0
Dense, 128 neuronas, ReLU	128	3,211,392
Dense, 3 neuronas, softmax	3	387
Total		**3,305,027**

La capa convolucional aplica a la imagen un conjunto de filtros pequeños cuyos valores se aprenden. Para un filtro K de tamaño $3 \times 3 \times C$ y un sesgo b, la salida en la posición (i, j) es

$$z_{i,j} = b + \sum_{c=1}^{C} \sum_{m=0}^{2} \sum_{n=0}^{2} K_{m,n,c}\, x_{i+m,\, j+n,\, c}, \qquad (1)$$

y sobre ella se aplica la función de activación ReLU, $\mathrm{ReLU}(z) = \max(0, z)$. Las primeras capas detectan rasgos simples, como bordes y texturas, y las siguientes los combinan en formas más complejas. La agrupación máxima de 2×2 conserva el mayor valor de cada ventana y reduce a la mitad el tamaño espacial, lo que disminuye el número de parámetros y da cierta tolerancia a pequeños desplazamientos.

La última capa tiene tres neuronas con activación softmax, que convierte los valores z_1, z_2, z_3 en probabilidades de cada clase:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{3} e^{z_j}}. \qquad (2)$$

Como las etiquetas son números enteros, se usa la entropía cruzada categórica dispersa, que penaliza la

probabilidad baja que el modelo asigna a la clase correcta y_i:

$$L = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,y_i}. \tag{3}$$

El entrenamiento utiliza el optimizador Adam con una tasa de aprendizaje de 0.001, lotes de 32 imágenes y 15 épocas. La tasa de aprendizaje y el tamaño de lote son los valores predeterminados de Keras, y el número de épocas y de bloques convolucionales se fijaron de antemano, sin ajustarlos. La profundidad se revisó más adelante comparando redes de 2 a 5 bloques (sección 5.9).

5.2. Evaluación del modelo base

La Figura 6 muestra la evolución del entrenamiento. La exactitud en entrenamiento llega a 100 % desde la época 8, y la pérdida de validación baja hasta 0.004 y se mantiene prácticamente estable (entre 0.0040 y 0.0044), por lo que no se observa el deterioro de la validación típico del sobreajuste.

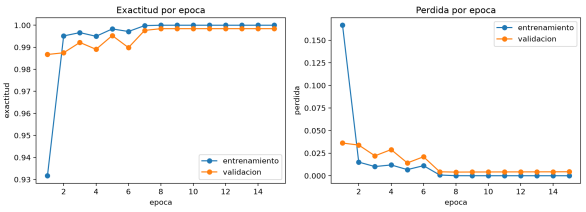

Figura 6. Exactitud y pérdida por época de la CNN base, en entrenamiento y validación.

En el conjunto de validación el modelo obtuvo una exactitud de 99.84 % (F1 macro de 0.9985) y en el de prueba de 99.53 % (F1 macro de 0.9953), con un F1 de 0.996, 0.993 y 0.997 para anime, cartoon y human, respectivamente. La matriz de confusión en prueba (Figura 16) muestra 6 errores en 1,281 imágenes: 3 de anime clasificadas como cartoon, 2 de cartoon clasificadas como human y 1 de human clasificada como cartoon.

Estos valores son muy altos, pero, como se muestra en la sección de predicciones con imágenes nuevas, solo describen el desempeño con imágenes parecidas a las del conjunto de datos.

5.3. Transferencia de aprendizaje

Como la CNN base solo acertó 2 de las 8 imágenes externas, se probó una red preentrenada. La transferencia de aprendizaje aprovecha las características visuales que aprendió una red entrenada con un conjunto de datos muy grande: las primeras capas detectan rasgos generales, como bordes y texturas, que sirven para muchas tareas, y solo las últimas capas se ajustan al problema nuevo. Gando et al. [6] aplican esta estrategia a un problema cercano, distinguir ilustraciones de fotografías: ajustan la capa totalmente conectada FC7 de AlexNet preentrenada con ImageNet y reinician la capa de salida (FC8), con el argumento de que su conjunto de datos (20,000 imágenes) es pequeño frente a ImageNet, y reportan una exactitud de 96.8 %, superior a la de las CNN propias que entrenaron desde cero con entre 1 y 5 capas convolucionales, la mejor de las cuales (la de 5 capas) alcanzó 93.0 %.

Se utilizó EfficientNetB0 [8] con los pesos preentrenados en ImageNet [9]. EfficientNet escala de forma conjunta la profundidad, el ancho y la resolución de la red, y la versión B0 tiene 4,049,571 parámetros sin su clasificador original. Se le quitó ese clasificador y se añadió una cabeza nueva, formada por una capa de agrupación promedio global y una capa densa de 3 neuronas con softmax (3,843 parámetros). Como se explicó en la sección de ETL, esta red recibe los píxeles en el rango $[0, 255]$ sin dividirlos entre 255.

El entrenamiento se hizo en dos fases. En la primera, la base se mantuvo congelada y solo se entrenó la cabeza, durante 8 épocas con Adam y una tasa de aprendizaje de 0.001. En la segunda se descongelaron las últimas 20 de las 238 capas de la base, dejando congeladas las capas de normalización por lotes, y se continuó el entrenamiento durante 10 épocas con una tasa de aprendizaje mucho menor (0.00001), para no alterar bruscamente los pesos preentrenados; en esta fase se entrenan 1,346,611 parámetros. El número de capas se fijó de antemano y en la sección 5.10 se compara con otras profundidades. Para poder atribuir los cambios a la red preentrenada, se usaron las mismas imágenes y particiones que en la CNN base, sin aumento de datos ni regularización adicional.

La Figura 7 muestra el entrenamiento. En validación el modelo alcanzó una exactitud de 99.92 % (F1 macro de 0.9992) y en prueba de 99.92 % (F1 macro de 0.9992), con un único error en prueba, una imagen de human clasificada como cartoon (Figura 16), frente a los 6 errores de la CNN base. Los resultados con imágenes externas se presentan en la sección de predicciones con imágenes nuevas.

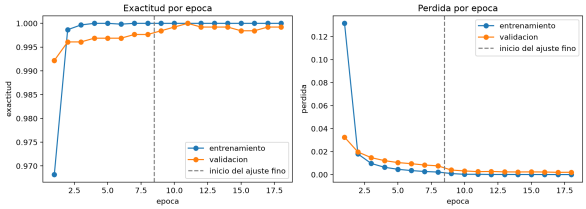

Figura 7. Exactitud y pérdida por época de EfficientNetB0. La línea punteada marca el inicio del ajuste fino.

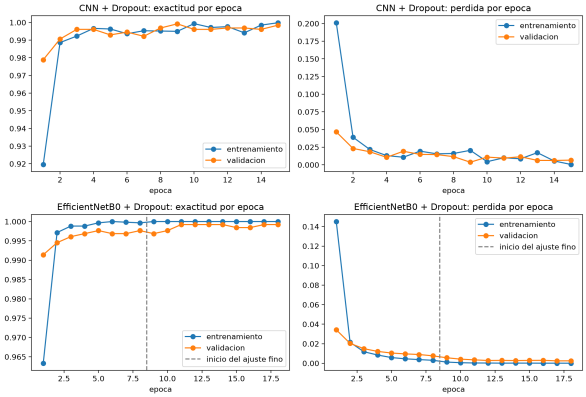

Figura 8. Exactitud y pérdida por época de los modelos con Dropout. En EfficientNetB0, la línea punteada marca el inicio del ajuste fino.

5.4. Regularización con Dropout

Ninguno de los dos modelos anteriores incluye regularización explícita. Dropout [10] apaga al azar una fracción de las neuronas de una capa en cada paso de entrenamiento: cada neurona se descarta con probabilidad r y las restantes se escalan por $1/(1-r)$. De esta forma la red no puede depender de neuronas individuales y se reduce la coadaptación entre ellas. Solo actúa durante el entrenamiento; al predecir se usan todas las neuronas.

Se probó por separado en cada arquitectura, con una sola tasa por modelo y sin ajustarla, para no sobreajustar a un conjunto externo tan pequeño. En la CNN se añadió con $r = 0.5$ después de la capa densa de 128 neuronas, que concentra el 97 % de los parámetros de la red; 0.5 es el valor habitual para capas densas. En EfficientNetB0 se añadió con $r = 0.2$ después de la agrupación promedio global, la misma tasa que usa el clasificador original de esa red. El resto del entrenamiento (datos, particiones, épocas, tasas de aprendizaje y fases) fue idéntico al de los modelos sin Dropout, de modo que la única diferencia es esa capa. Cada modelo se entrenó una sola vez.

La Figura 8 muestra el entrenamiento. En la CNN, el Dropout tuvo el efecto esperado sobre las curvas: la pérdida de entrenamiento al final de las 15 épocas es de 5.5×10^{-4}, frente a 6.3×10^{-6} sin Dropout, y las curvas son más irregulares, porque cada paso usa una red distinta. Sin embargo, esto no se tradujo en mejores métricas de validación: la exactitud fue la misma (99.84 %) y la pérdida final de validación fue algo mayor (0.0067 frente a 0.0044). En EfficientNetB0 los dos entrenamientos fueron prácticamente iguales (pérdida final de validación de 0.0025 con Dropout y 0.0020 sin él).

En el conjunto de prueba, la CNN con Dropout cometió 5 errores (exactitud de 99.61 %) frente a 6 sin Dropout, y EfficientNetB0 con Dropout no cometió ninguno (100 %) frente a 1 sin él (Figura 9). Son diferencias de una sola imagen en 1,281, que no permiten afirmar una mejora, y como cada modelo se entrenó una vez, también pueden deberse a la aleatoriedad del entrenamiento.

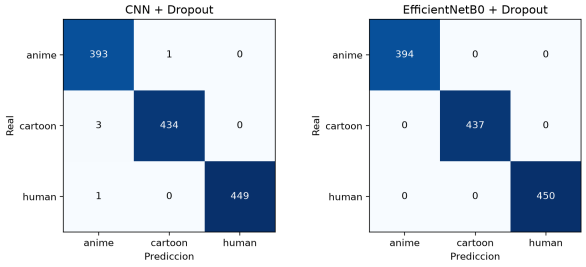

Figura 9. Matrices de confusión de los modelos con Dropout en el conjunto de prueba.

Con las imágenes externas, ningún modelo cambió su clase predicha en ninguna de las 8 imágenes (Tabla 4): la CNN con Dropout sigue acertando 2 de 8 (25.0 %) y EfficientNetB0 con Dropout 4 de 8 (50.0 %), con los mismos aciertos y errores que sin Dropout. La confianza cambió poco (por ejemplo, la viñeta de manga, clasificada erróneamente como cartoon por EfficientNetB0, pasó de 94 % a 98 % de confianza) y los errores de la CNN siguen teniendo una confianza cercana a 100 %.

Cuadro 4. Predicciones de los modelos con Dropout sobre imágenes externas (entre paréntesis, la confianza)

Imagen	Real	CNN + Dropout	EfficientNetB0 + Dropout
`Caricaturas2.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)
`caricatura1.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)
`pruebaAnime.jpg`	anime	cartoon (100 %)	cartoon (100 %)
`pruebaAnime2.jpg`	anime	cartoon (100 %)	cartoon (100 %)
`pruebaAnime3.jpg`	anime	cartoon (100 %)	cartoon (98 %)
`pruebaAnime4.jpg`	anime	cartoon (100 %)	anime (99 %)
`pruebaPersona.jpg`	human	cartoon (100 %)	cartoon (97 %)
`pruebaPersona2.jpg`	human	cartoon (100 %)	human (82 %)

Este resultado es coherente con la naturaleza del problema. Dropout combate el sobreajuste, y en estos modelos la exactitud de entrenamiento y la de validación ya eran casi iguales, de modo que no había una brecha que cerrar. Reducir la coadaptación entre neuronas no corrige que las imágenes externas sean distintas a las del conjunto de datos, que es donde se concentra el error. Dropout se probó y se documenta, pero no aporta una mejora medible en este caso.

5.5. Tratamiento de los datos: estirar y aumento de datos

Los experimentos anteriores sugieren que el problema no está tanto en el modelo como en la poca diversidad de los datos de entrenamiento. Por ello se probaron dos cambios en el tratamiento de los datos, sobre EfficientNetB0 (el mejor modelo hasta ahora, sin Dropout, que no aportó mejora) y con el mismo entrenamiento en dos fases. Para poder explicar cuál de los dos cambios funciona, se probaron por separado y juntos, lo que da las cuatro variantes de la Tabla 5.

Cuadro 5. Variantes de tratamiento de los datos (todas con EfficientNetB0)

Variante	Redimensionado	Aumento de datos
EfficientNetB0 (anterior)	padding	no
+ estirar	estirar	no
+ aumento (padding)	padding	sí
+ estirar y aumento	estirar	sí

Estirar. En lugar de conservar la proporción y rellenar con bandas, la imagen se redimensiona directamente a 128×128, con el mismo filtro de Lanczos. Esto elimina las bandas de relleno, que en el conjunto de datos solo aparecían en las imágenes de cartoon no cuadradas y podían servir de atajo, a cambio de deformar las imágenes que no son cuadradas.

Aumento de datos. Consiste en aplicar transformaciones aleatorias a las imágenes de entrenamiento, distintas en cada época, de modo que el modelo vea muchas variantes de cada imagen. Se aplica imagen por imagen y solo en entrenamiento; la validación y la prueba no se modifican. Las transformaciones se eligieron para atacar los atajos identificados en el análisis exploratorio (composición y proporción, color y nitidez):

- volteo horizontal;
- rotación de hasta 11 grados;
- zoom de hasta 25 % en cada eje de forma independiente, lo que también varía la proporción de la imagen;
- traslación de hasta 10 %;
- cambios de brillo, contraste y saturación;
- conversión a escala de grises, con probabilidad de 0.15 (por ejemplo, para manga o fotografías antiguas);
- baja resolución, con probabilidad de 0.5: la imagen se reduce a entre 35 % y 100 % de su tamaño y se vuelve a ampliar, para que la nitidez no delate la clase.

Los huecos que dejan la rotación, el zoom y la traslación se rellenan reflejando la imagen. La Figura 10 muestra ejemplos.

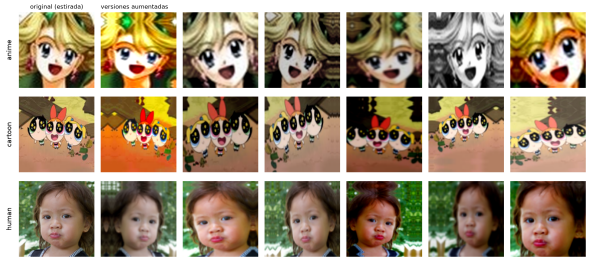

Figura 10. Imagen original (estirada) de cada clase y seis versiones aumentadas.

La Figura 11 muestra el entrenamiento. Sin aumento de datos, la exactitud de entrenamiento llega a 100 % y la de validación queda por debajo. Con aumento, las curvas de entrenamiento y de validación casi se superponen y la exactitud de entrenamiento no llega a 100 % (99.87 % y 99.75 % al final), que es el efecto regularizador esperado.

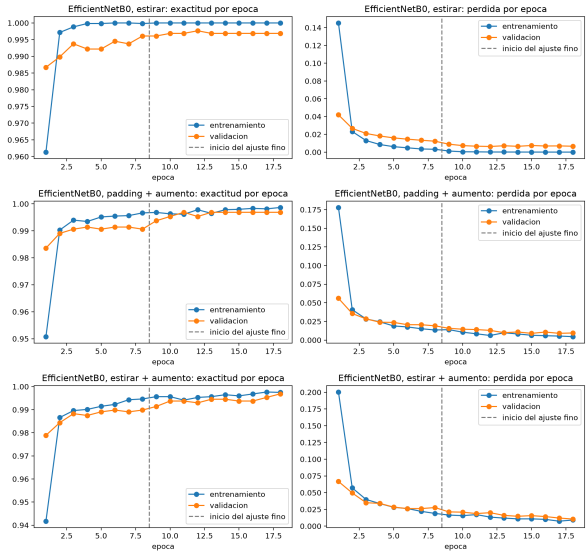

Figura 11. Exactitud y pérdida por época de las tres variantes nuevas. La línea punteada marca el inicio del ajuste fino.

En validación y prueba, las tres variantes nuevas rinden ligeramente peor que el modelo original (Tabla 11): 99.69 % frente a 99.92 % en validación, y entre 2 y 3 errores en prueba frente a 1. Es el costo esperable de entrenar con datos más variados, y son diferencias de una o dos imágenes.

Con las imágenes externas (Tabla 6), estirar por sí solo no cambia ningún acierto (4 de 8), pero reduce la seguridad de los errores: la ilustración anime `pruebaAnime2.jpg`, clasificada como cartoon con 100 % de confianza, baja a 84 %. El aumento de datos sí corrige una imagen, la viñeta de manga en blanco y negro (`pruebaAnime3.jpg`), con lo que se llega a 5 de 8 (exactitud de 62.5 % y F1 macro de 0.635), tanto con padding como estirando. Esto es coherente con que el aumento incluye imágenes en escala de grises, aunque este experimento no lo comprueba. Combinar ambos cambios no mejora el resultado respecto al aumento solo (5 de 8) y reduce todavía más la confianza de los errores (54 % en `pruebaAnime2.jpg`).

Cuadro 6. Predicciones de las variantes de tratamiento de datos sobre imágenes externas (entre paréntesis, la confianza)

Imagen	Real	Anterior	+ estirar	+ aumento	+ estirar y aumento
`Caricaturas2.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)
`caricatura1.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)	cartoon (99 %)
`pruebaAnime.jpg`	anime	cartoon (100 %)	cartoon (90 %)	cartoon (91 %)	cartoon (85 %)
`pruebaAnime2.jpg`	anime	cartoon (100 %)	cartoon (84 %)	cartoon (91 %)	cartoon (54 %)
`pruebaAnime3.jpg`	anime	cartoon (94 %)	cartoon (99 %)	anime (91 %)	anime (69 %)
`pruebaAnime4.jpg`	anime	anime (97 %)	anime (89 %)	anime (87 %)	anime (95 %)
`pruebaPersona.jpg`	human	cartoon (98 %)	cartoon (100 %)	cartoon (99 %)	cartoon (100 %)
`pruebaPersona2.jpg`	human	human (81 %)	human (100 %)	human (84 %)	human (96 %)

En todas las variantes siguen fallando las dos ilustraciones anime de estilo moderno y el modelo anatómico, y todos los errores se clasificaron como cartoon. La mejora es modesta y poco concluyente: es un acierto más en 8 imágenes, cada modelo se entrenó una sola vez y las diferencias en validación y prueba son de una o dos imágenes. Una posible explicación de que el aumento de datos no corrija las ilustraciones anime modernas, que no se comprobó, es que en el conjunto de entrenamiento las imágenes de anime son solo rostros recortados: transformaciones como el zoom o la traslación varían lo que se ve de esos rostros, pero no crean personajes con cuerpo, fondos ni estilos que el conjunto no contiene. Resolverlo requeriría más diversidad en los propios datos de entrenamiento.

5.6. Aumento de datos dirigido a la falla

Las ilustraciones anime modernas seguían sin acertarse. Como hipótesis, que no se comprobó, el anime del conjunto de entrenamiento son rostros que ocupan todo el cuadro, mientras que las imágenes externas tienen cuerpo, fondo o estilo distinto. Para simular esa diferencia se añadieron tres transformaciones al aumento anterior (Figura 12):

- **alejar la imagen:** se reduce a entre 40 % y 80 % del lado y se coloca al azar sobre un fondo plano del color del borde, para que el rostro quede como parte de una escena;
- **recorte agresivo:** se recorta una región de entre 30 % y 80 % del área, con proporción entre 3:4 y 4:3, y se amplía;
- **borrado aleatorio:** se tapa un rectángulo de entre 2 % y 25 % del área con ruido.

Cada imagen recibe un alejamiento o un recorte (35 % de probabilidad cada uno) y, con 50 % de probabilidad, un borrado. Se usaron imágenes estiradas, para que los fondos planos del alejamiento no se confundan con las barras de relleno del padding, y el mismo entrenamiento en dos fases. Una ejecución exploratoria previa con padding, que no se conservó, obtuvo 5 de 8 en las imágenes externas (F1 macro de 0.611), 99.30 % en validación y 99.38 % en prueba.

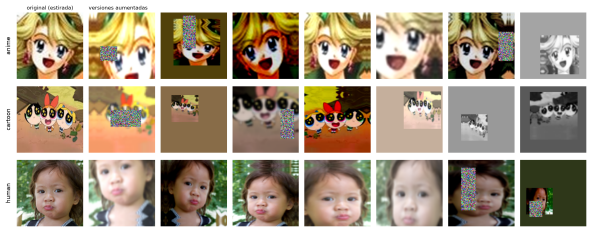

Figura 12. Una imagen por clase (columna izquierda) y siete versiones con el aumento dirigido.

El resultado no mejora al del aumento anterior: 4 de 8 imágenes externas (exactitud de 50.0 % y F1 macro de 0.490), frente a 5 de 8 con el aumento anterior, y 9 errores en prueba frente a 3 (Tabla 11). Cambió además cuáles imágenes se aciertan: la viñeta de manga `pruebaAnime3.jpg`, que el aumento anterior sí acertaba, pasa a cartoon, y `pruebaAnime2.jpg` se clasifica ahora como human (60 %) y no como cartoon. Siguen sin acertarse las ilustraciones anime modernas, y por primera vez un error no se clasifica como cartoon.

5.7. Ablación del aumento dirigido

Como el aumento dirigido junta tres transformaciones, no se sabe cuál produce el cambio. Se entrenó una variante por cada una, partiendo del aumento anterior con imágenes estiradas y añadiendo solo esa transformación, con la misma probabilidad que en el aumento dirigido completo. El entrenamiento, la semilla y las imágenes externas son los mismos, y cada variante se entrenó una sola vez. La Tabla 7 muestra los resultados por imagen externa.

Cuadro 7. Predicciones por imagen externa de la ablación (entre paréntesis, la confianza). Base: aumento anterior con imágenes estiradas; Dir.: las tres transformaciones juntas

Imagen	Real	Base	Alejar	Recortar	Borrar	Dir.
`Caricaturas2.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)
`caricatura1.jpg`	cartoon	cartoon (99 %)	cartoon (88 %)	cartoon (95 %)	cartoon (99 %)	cartoon (94 %)
`pruebaAnime.jpg`	anime	cartoon (85 %)	anime (77 %)	cartoon (77 %)	cartoon (94 %)	cartoon (81 %)
`pruebaAnime2.jpg`	anime	cartoon (54 %)	human (57 %)	cartoon (62 %)	human (67 %)	human (60 %)
`pruebaAnime3.jpg`	anime	anime (69 %)	cartoon (61 %)	anime (84 %)	anime (80 %)	cartoon (55 %)
`pruebaAnime4.jpg`	anime	anime (95 %)	human (64 %)	anime (82 %)	anime (88 %)	anime (72 %)
`pruebaPersona.jpg`	human	cartoon (100 %)	cartoon (99 %)	cartoon (99 %)	cartoon (96 %)	cartoon (94 %)
`pruebaPersona2.jpg`	human	human (96 %)	human (99 %)	human (96 %)	human (100 %)	human (97 %)

- **Alejar** es la única que acierta una imagen que el modelo base no acertaba (`pruebaAnime.jpg`), pero pierde la viñeta de manga y el rostro pixelado. Es también la que más empeora la validación (99.14 %) y la prueba (7 errores frente a 3), con 4 de 8 imágenes externas.
- **Recortar y borrar** aciertan exactamente las mismas imágenes que el modelo base (5 de 8) y no cambian el resultado; borrar solo cambia a qué clase se asigna un error (`pruebaAnime2.jpg` pasa de cartoon a human).
- **Las tres juntas** dan los mismos 4 de 8 que alejar sola, de modo que el efecto negativo parece venir de alejar y no de la combinación.

Ninguna técnica superó al modelo base. La hipótesis de los rostros recortados recibe un apoyo débil: alejar es la única que acierta una ilustración anime moderna, pero a cambio de otras dos imágenes. Cada variante se entrenó una sola vez y con 8 imágenes una diferencia de una imagen no se distingue del azar del entrenamiento, que no se midió.

5.8. Imágenes de 224 × 224 píxeles

Hasta aquí todos los modelos recibían imágenes de 128 × 128, mientras que EfficientNetB0 fue diseñada para 224 × 224 y a menor resolución se pierde detalle en las ilustraciones. Se repitió el modelo con imágenes estiradas y el aumento de datos anterior cambiando solo la resolución: las imágenes se transformaron de nuevo desde los archivos originales a 224 × 224, con la misma función y el mismo remuestreo Lanczos, y se entrenó con el mismo procedimiento en dos fases. Las imágenes externas también se evaluaron a 224 × 224.

Con 224 píxeles la exactitud en validación sube de 99.69 % a 99.84 % y la de prueba de 99.77 % a 99.92 % (de 3 errores a 1), la mejor de las variantes con imágenes estiradas. En las imágenes externas el total no cambia (5 de 8, F1 macro de 0.635), pero sí cuáles se aciertan: `pruebaAnime2.jpg`, la ilustración de un personaje, se clasifica por primera vez como anime (65 %), y la manga `pruebaAnime3.jpg` sube a 95 % de confianza, mientras que el rostro pixelado `pruebaAnime4.jpg` pasa a cartoon (98 %). Todos los errores vuelven a clasificarse como cartoon. Es un solo entrenamiento y 8 imágenes, así que la mejora en validación y prueba es de dos imágenes y la ganancia y la pérdida en las externas no son concluyentes; con esos datos, aumentar la resolución no resolvió la falla de fondo.

5.9. Profundidad de la CNN propia

La CNN base tiene 3 bloques convolucionales, un valor que se fijó de antemano. Para revisarlo se siguió la idea de Gando et al. [6], que en un problema cercano entrenaron desde cero CNN propias con entre 1 y 5 capas convolucionales y compararon su exactitud. Se entrenó la misma red con 2, 3, 4 y 5 bloques, en los que cada bloque añade una convolución de 3 × 3 con el do-

ble de filtros que el anterior (32, 64, 128, 256 y 512) y una agrupación máxima de 2×2; todo lo demás (padding, sin regularización ni aumento de datos, Adam con 0.001, lotes de 32 y 15 épocas) es igual al de la CNN base.

Como la validación está saturada y una diferencia de una imagen puede deberse al azar del entrenamiento, cada profundidad se entrenó con tres semillas (42, 43 y 44), incluida la de 3 bloques, que es la referencia. La regla de decisión se fijó antes de entrenar: se adopta una profundidad solo si aumenta en al menos una imagen el promedio de aciertos en las imágenes externas respecto a la referencia. La selección usa únicamente ese conjunto porque validación y prueba comparten sesgos con el entrenamiento. Validación y prueba se reportan como métricas descriptivas, no como criterios de selección.

Cuadro 8. Comparación de profundidades de la CNN propia (media y desviación estándar sobre 3 semillas)

Bloques	Parámetros	Exactitud de validación	Pérdida de validación	Aciertos externos
2	7,392,707	0.9977 ± 0.0013	0.0059	2 de 8
3 (referencia)	3,305,027	0.9984 ± 0.0000	0.0071	2 de 8
4	1,568,579	0.9951 ± 0.0019	0.0164	2 de 8
5	1,831,235	0.9945 ± 0.0034	0.0230	2 de 8

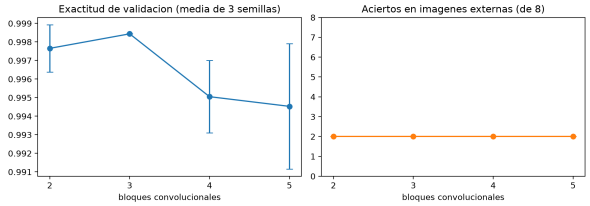

Figura 13. Exactitud de validación y aciertos en imágenes externas según el número de bloques convolucionales (media y desviación estándar sobre 3 semillas).

La Tabla 8 y la Figura 13 muestran el resultado. La red de 3 bloques obtuvo la mayor exactitud media de validación (99.84 % con las tres semillas; con la semilla 42 reproduce exactamente la CNN base), mientras que las redes de 4 y 5 bloques quedaron algo por debajo y con más variación entre semillas, y la de 2 bloques quedó en un punto intermedio. En las imágenes externas no hubo ninguna diferencia: las 12 redes entrenadas acertaron solo las dos caricaturas, porque todas clasificaron las 8 imágenes como cartoon, con una confianza de al menos 92.9 %. Ninguna profundidad cumple la regla, por lo que se mantienen los 3 bloques.

A diferencia de Gando et al., en este caso más profundidad no mejoró la red. Sus redes se entrenaron con imágenes de 256×256, normalización por lotes y 100 épocas, con otros datos, por lo que los resultados no son directamente comparables. Que el error externo sea el mismo con cualquier profundidad sugiere, sin comprobarlo, que la falla de la CNN no depende de su capacidad sino de los datos con que se entrena.

5.10. Profundidad del ajuste fino de EfficientNetB0

En el ajuste fino de EfficientNetB0 se descongelaron las últimas 20 de las 238 capas de la base, un número que también se fijó de antemano. En la práctica, esas 20 capas son el bloque `block7a`, la convolución final `top_conv` y la última convolución de `block6d`. Para revisarlo se siguió el procedimiento de Tajbakhsh et al. [7], que eligen la profundidad del ajuste fino de forma incremental: empiezan por la última capa y van incorporando capas hacia atrás mientras el desempeño mejore. En sus experimentos con imágenes médicas, ni el ajuste superficial ni el profundo fue la mejor opción en todas las aplicaciones, y proponen este recorrido por capas como una forma práctica de encontrar la adecuada para cada caso.

Se aplicó ese procedimiento al modelo con imágenes estiradas y aumento de datos, a 128 píxeles, cambiando solo la parte de la base que se descongela en la segunda fase. Las capas de normalización por lotes siguieron congeladas y el resto del entrenamiento fue igual. Como EfficientNetB0 está organizada en bloques, los pasos se definieron por bloques. De menos a más profundo, son: entrenar solo la cabeza, sin descongelar la base; descongelar desde `top_conv`; desde `block7a`; desde `block6a`; y así sucesivamente hasta la base completa.

La configuración de 20 capas es la referencia, y se usaron la misma regla de decisión y las mismas tres semillas que en la CNN. Durante el recorrido, cada paso se comparó con el mejor anterior usando solo el promedio de aciertos externos: un paso mejora si acierta más imágenes. El recorrido se detiene tras dos pasos seguidos sin mejora.

Cuadro 9. Profundidad del ajuste fino de EfficientNetB0 (media y desviación estándar sobre 3 semillas)

Parte ajustada	Parámetros entrenables	Exactitud de validación	Pérdida de validación	Aciertos externos
Solo la cabeza (seleccionada)	3,843	0.9904 ± 0.0010	0.0251	6.00 ± 0.00
Desde `top_conv`	413,443	0.9924 ± 0.0007	0.0207	5.67 ± 0.47
Desde `block7a`	1,125,427	0.9956 ± 0.0010	0.0127	5.00 ± 0.00
Últimas 20 capas (referencia)	1,346,611	0.9948 ± 0.0004	0.0136	5.00 ± 0.00

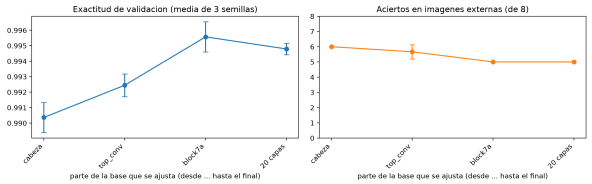

Figura 14. Exactitud de validación y aciertos en imágenes externas según la parte de EfficientNetB0 que se ajusta (media y desviación estándar sobre 3 semillas).

La Tabla 9 y la Figura 14 muestran el resultado. Entrenar solo la cabeza fue el mejor paso por aciertos externos, con 6 de 8 imágenes en las tres semillas, una más que la referencia. La imagen adicional es la ilustración anime horizontal (`pruebaAnime.jpg`), que antes solo había acertado la variante con alejar. Aquí se acierta sin perder las demás, aunque con poca confianza (entre 52 % y 71 %). Los dos pasos siguientes no superaron este resultado, por lo que el recorrido se detuvo en `block7a` y no se entrenaron los bloques anteriores, de `block6a` hacia atrás.

La configuración seleccionada por la regla actualizada es entrenar solo la cabeza. Su exactitud media de validación es 99.04 %, frente a 99.48 % para la referencia de 20 capas; la métrica se reporta como descripción y no interviene en la decisión. Para la interfaz se guardó una nueva ejecución de esta configuración: alcanzó 98.98 % en validación, 99.45 % en prueba y 6 de 8 imágenes externas. El conjunto externo tiene solo 8 imágenes, por lo que una diferencia de un acierto no es concluyente. Con la semilla 42, la referencia no reproduce exactamente el modelo de la sección de aumento de datos (99.53 % frente a 99.69 % en validación, ambos con 5 de 8 externas), probablemente porque aquí el conjunto aumentado se crea en otro punto de la secuencia de números aleatorios.

El resultado es coherente con la observación de Tajbakhsh et al. de que no hay una profundidad de ajuste que sea la mejor en todos los casos: en este problema, la profundidad que mejor se ajusta al conjunto de datos no es la que mejor generaliza a las imágenes externas. Una posible explicación, que no se comprobó, es que al ajustar más capas la red adapta las características de ImageNet a los sesgos del conjunto de entrenamiento. La diferencia se repite en las tres semillas, pero depende de una sola imagen de 8, por lo que no es concluyente.

6. Predicciones con imágenes nuevas

Las métricas de validación y de prueba se calculan con imágenes del mismo conjunto de datos, por lo que comparten con el entrenamiento los sesgos descritos en el análisis exploratorio (resolución, color y composición). Para comprobar si el modelo generaliza, se evaluó con imágenes que no pertenecen al conjunto de datos, con el mismo preprocesado que se usó al entrenar.

6.1. Conjunto de imágenes externas

Se reunieron 8 imágenes, y la clase de cada una se deduce del prefijo de su nombre de archivo. Antes de evaluar, se compararon por contenido (resumen MD5) con las imágenes del conjunto de datos y se comprobó que ninguna es copia de ellas, de modo que el modelo no pudo verlas al entrenar. Son 4 de anime, 2 de cartoon y 2 de human. Las imágenes se obtuvieron de forma aleatoria de internet; no se verificaron sus licencias ni se registró su fuente. Se usan únicamente con fines académicos, para evaluar el desempeño de los modelos y ilustrar los resultados de este reporte, y no con fines comerciales. Los derechos de cada imagen pertenecen a sus respectivos autores.

Es un conjunto muy pequeño, por lo que permite observar patrones pero no estimar el desempeño con precisión. Además, una de las imágenes de human es un modelo anatómico de musculatura en 3D y no una fotografía, es decir, no corresponde al tipo de imagen de esa clase en el conjunto de entrenamiento.

6.2. Resultados con imágenes externas

La Tabla 10 muestra las predicciones de los dos modelos y la Figura 15 las de la CNN base. La CNN base acertó 2 de las 8 imágenes (exactitud de 25.0 % y F1 macro de 0.133), frente a 99.53 % de exactitud en el conjunto de prueba.

Cuadro 10. Predicciones sobre imágenes externas (entre paréntesis, la confianza de EfficientNetB0)

Imagen	Descripción	Real	CNN base	EfficientNetB0
`Caricaturas2.jpg`	Collage de personajes de caricaturas	cartoon	cartoon	cartoon (100 %)
`caricatura1.jpg`	Caricatura dibujada de una persona	cartoon	cartoon	cartoon (100 %)
`pruebaAnime.jpg`	Ilustración anime horizontal	anime	cartoon	cartoon (100 %)
`pruebaAnime2.jpg`	Ilustración anime de personaje	anime	cartoon	cartoon (100 %)
`pruebaAnime3.jpg`	Viñeta de manga en blanco y negro	anime	cartoon	cartoon (94.4 %)
`pruebaAnime4.jpg`	Rostro en arte pixelado	anime	cartoon	anime (97.1 %)
`pruebaPersona.jpg`	Modelo anatómico 3D	human	cartoon	cartoon (97.6 %)
`pruebaPersona2.jpg`	Fotografía de un hombre	human	cartoon	human (80.7 %)

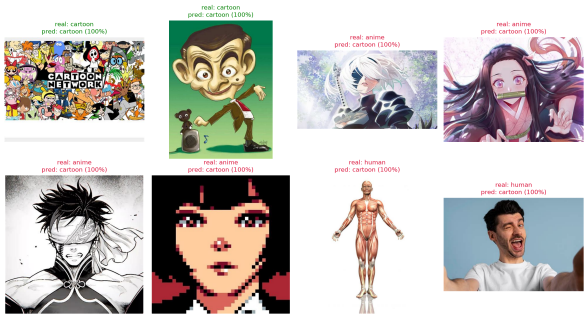

Figura 15. Predicciones de la CNN base sobre las imágenes externas (en verde los aciertos y en rojo los errores).

En la CNN base, las dos imágenes acertadas son las de cartoon. Los 6 errores se clasificaron como cartoon, ninguno como anime ni como human, y todas las predicciones tienen una confianza de al menos 99.999 %, es decir, el modelo se equivoca con total seguridad.

Para saber si este resultado se debe al relleno (padding) de las imágenes no cuadradas, se repitió la evaluación con dos variantes sin barras de relleno: estirando la imagen a un cuadrado y recortando su centro. Las predicciones fueron las mismas en las tres variantes, por lo que el padding no explica estos errores.

El resultado indica que la exactitud casi perfecta en validación y prueba no se mantiene con imágenes de otro origen. Una posible explicación, que este experimento no comprueba, es que el modelo aprendió una noción estrecha de las clases anime y human (rostros recortados de un estilo y una toma particulares) y que cartoon, la clase más heterogénea según el análisis exploratorio, funciona como clase por defecto para todo lo que no se parece a esos ejemplos. Esto motivó probar una red preentrenada y las demás mejoras, que se comparan también con este conjunto externo y no solo con el de validación.

Con EfficientNetB0 se acertaron 4 de las 8 imágenes (exactitud de 50.0 % y F1 macro de 0.522). Además de las dos caricaturas, acertó el rostro en arte pixelado (anime) y la fotografía del hombre (human), esta última con la confianza más baja de todas (80.7 %). Siguió fallando con las dos ilustraciones anime de estilo moderno, la viñeta de manga y el modelo anatómico, y todos sus errores se clasificaron de nuevo como cartoon, aunque con una confianza algo menor en algunos casos (94.4 % y 97.6 %). Con solo 8 imágenes, la diferencia de 2 aciertos no es concluyente: sugiere que la transferencia de aprendizaje ayuda algo, pero no resuelve el problema. Una posible explicación, que no se comprobó, es que el clasificador se entrena con ejemplos de anime y human muy estrechos (rostros recortados de un estilo particular), por lo que las características preentrenadas por sí solas no corrigen la falta de diversidad de los datos.

6.3. Interfaz web

Las predicciones también pueden realizarse mediante una interfaz web. Un servidor en Flask expone los modelos entrenados, y una página (HTML, CSS y JavaScript) permite elegir el modelo en un selector, subir una imagen y ver la clase predicha, la probabilidad de cada clase y la imagen tal como la ve el modelo. Al cambiar de modelo con una imagen ya cargada, la predicción se recalcula. El servidor usa el mismo módulo de preprocesado que el entrenamiento, y a cada modelo le aplica el redimensionado con que se entrenó (padding o estirar), para que reciba lo mismo que vio al aprender.

7. Resultados

La Tabla 11 resume la exactitud y el F1 macro de los modelos en los tres conjuntos de evaluación, y la Figura 16 muestra sus matrices de confusión en el conjunto de prueba.

Cuadro 11. Exactitud y F1 macro por modelo y conjunto de evaluación

	Validación		Prueba		Externas (8)	
Modelo	Exactitud	F1	Exactitud	F1	Exactitud	F1
CNN base (desde cero)	0.9984	0.9985	0.9953	0.9953	0.2500	0.1333
CNN + Dropout	0.9984	0.9985	0.9961	0.9960	0.2500	0.1333
EfficientNetB0 (transfer learning)	0.9992	0.9992	0.9992	0.9992	0.5000	0.5222
EfficientNetB0 + Dropout	0.9992	0.9992	1.0000	1.0000	0.5000	0.5222
EfficientNetB0 + estirar	0.9969	0.9968	0.9984	0.9984	0.5000	0.5222
EfficientNetB0 + aumento (padding)	0.9969	0.9969	0.9984	0.9984	0.6250	0.6349
EfficientNetB0 + estirar y aumento	0.9969	0.9968	0.9977	0.9976	0.6250	0.6349
EfficientNetB0 + estirar y aumento dirigido	0.9945	0.9945	0.9930	0.9929	0.5000	0.4905
EfficientNetB0 + estirar y alejar	0.9914	0.9914	0.9945	0.9945	0.5000	0.4889
EfficientNetB0 + estirar y recortar	0.9961	0.9961	0.9969	0.9969	0.6250	0.6349
EfficientNetB0 + estirar y borrar	0.9961	0.9961	0.9977	0.9976	0.6250	0.6111
EfficientNetB0 + estirar y aumento (224 px)	0.9984	0.9985	0.9992	0.9992	0.6250	0.6349

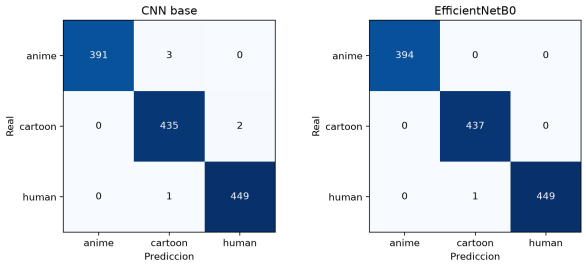

Figura 16. Matrices de confusión de la CNN base y de EfficientNetB0 en el conjunto de prueba.

En validación y en prueba todos los modelos superan el 99.5 % y las diferencias entre ellos son de una a tres imágenes, por lo que esos conjuntos no permiten distinguir entre modelos. Con las imágenes externas, en

cambio, el desempeño cae de forma notable y sí aparecen diferencias: 25.0 % en la CNN (con o sin Dropout), 50.0 % en EfficientNetB0 (con o sin Dropout, o estirando) y 62.5 % con aumento de datos (con padding o estirando). Dropout no cambió ninguna predicción externa; el aumento dirigido bajó a 50.0 % con más errores en prueba, entrenar a 224 píxeles mejoró validación y prueba pero dejó en 62.5 % el resultado externo, y la CNN acertó las mismas 2 imágenes con 2, 3, 4 o 5 bloques. En EfficientNetB0, entrenar solo la cabeza obtuvo el mejor resultado externo, con 6 de 8 aciertos; es la configuración seleccionada por el criterio externo, aunque su validación fue menor. La transferencia de aprendizaje y el aumento de datos mejoraron el resultado externo en 2 y en 1 de las 8 imágenes, respectivamente, lo cual no es concluyente con tan pocas imágenes. Ningún modelo acertó a la vez las dos ilustraciones anime de estilo moderno. Por eso, la evaluación con imágenes externas es la que se usa para comparar los modelos.

8. Conclusiones

Se construyó un clasificador de imágenes en tres clases (anime, cartoon y human) y se comparó una CNN propia con transferencia de aprendizaje y con varias modificaciones: Dropout, estirar las imágenes, aumento de datos (general y dirigido), mayor resolución y distintas profundidades de la CNN y del ajuste fino. Las conclusiones son las siguientes.

- **La exactitud en validación y prueba no distingue entre los modelos.** Todos superan el 99.5 % y difieren en una a seis imágenes de 1,281. Además, el análisis exploratorio mostró que las clases difieren en resolución, peso de archivo, color y composición, y la validación y la prueba comparten esos sesgos con el entrenamiento, por lo que una cifra alta ahí no garantiza que el modelo haya aprendido el contenido de las imágenes.
- **Los modelos no generalizan bien a imágenes de otro origen.** En las 8 imágenes externas, la CNN base acierta 2 (25.0 %) y el modelo seleccionado, con EfficientNetB0 y solo la cabeza entrenada, acierta 6 (75.0 %). Los errores se concentran en la clase cartoon y ocurren con una confianza alta, sobre todo en ilustraciones anime de estilo moderno; ningún modelo acertó a la vez las dos de este tipo.
- **Qué ayudó.** La transferencia de aprendizaje con EfficientNetB0 pasó de 2 a 4 aciertos externos y el aumento de datos añadió uno más (la viñeta de manga en blanco y negro). Estirar las imágenes no cambió los aciertos, pero redujo la confianza de algunos errores.
- **Qué no ayudó.** Dropout no cambió ninguna predicción externa en ninguna de las dos arquitecturas. El aumento dirigido a los rostros recortados tampoco superó al aumento anterior (4 de 8 frente a 5 de 8, con más errores en prueba), y al separarlo, ni recortar ni borrar cambiaron el resultado y alejar fue la única que acertó una imagen nueva, pero perdió otras dos. Entrenar a 224 píxeles redujo los errores en prueba de 3 a 1, pero no cambió el total de aciertos externos (5 de 8). Cambiar la profundidad de la CNN (de 2 a 5 bloques, con 3 semillas cada una) tampoco: las 12 redes clasificaron las 8 imágenes externas como cartoon. En EfficientNetB0, entrenar solo la cabeza mejoró los aciertos externos de 5 a 6 (en las tres semillas) y es la configuración seleccionada; su exactitud de validación bajó de 99.48 % a 99.04 %, una métrica que no intervino en la decisión. Esto es coherente con que el problema no sea un sobreajuste clásico, ya que la exactitud de entrenamiento y la de validación eran casi iguales.
- **Alcance de estos resultados.** El conjunto externo tiene solo 8 imágenes y, salvo en las comparaciones de profundidad, cada modelo se entrenó una vez, por lo que las diferencias de uno o dos aciertos no son concluyentes. La explicación de que el anime de entrenamiento sean solo rostros recortados, y por eso fallen las ilustraciones con cuerpo, fondo o estilo distinto, es una hipótesis que no se comprobó.

Como trabajo futuro, lo más probable es que el mayor avance venga de ampliar y diversificar los datos de entrenamiento (por ejemplo, anime con cuerpo completo y fondos, y fuentes distintas para cada clase), y de evaluar con un conjunto externo más grande y con varias repeticiones del entrenamiento para poder confirmar diferencias pequeñas.

Referencias

[1] hadiepratamatulili, "Anime vs Cartoon vs Human," Kaggle. [Online]. Available: `https://www.kaggle.com/datasets/hadiepratamatulili/anime-vs-cartoon-vs-human` [Accessed: Oct. 26, 2026].

[2] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: `https://www.tensorflow.org/`

[3] F. Chollet et al., "Keras," 2015. [Online]. Available: `https://keras.io`

[4] Pillow (PIL Fork), "2.7.0 release notes," Pillow Documentation. [Online]. Available:

https://pillow.readthedocs.io/en/stable/releasenotes/2.7.0.html

[5] G. Parmar, R. Zhang and J.-Y. Zhu, "On aliased resizing and surprising subtleties in GAN evaluation," arXiv:2104.11222, 2021. [Online]. Available: https://arxiv.org/abs/2104.11222

[6] G. Gando, T. Yamada, H. Sato, S. Oyama and M. Kurihara, "Fine-tuning deep convolutional neural networks for distinguishing illustrations from photographs," *Expert Systems with Applications*, vol. 66, pp. 295–301, 2016, doi: 10.1016/j.eswa.2016.08.057.

[7] N. Tajbakhsh, J. Y. Shin, S. R. Gurudu, R. T. Hurst, C. B. Kendall, M. B. Gotway and J. Liang, "Convolutional neural networks for medical image analysis: Full training or fine tuning?," *IEEE Transactions on Medical Imaging*, vol. 35, no. 5, pp. 1299–1312, 2016, doi: 10.1109/TMI.2016.2535302.

[8] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.

[9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.